

Donation Auth Module

Хандивын системийн хэрэглэгчийн бүртгэл, нэвтрэлт, OTP баталгаажуулалт, JWT authentication, refresh token болон role-based authorization модуль.

Tech Stack

Node.js, Express, MongoDB, Mongoose

Security Focus

bcrypt, JWT, OTP policy, refresh token rotation

Project Type

Backend Authentication & Authorization Module

Test Result

1 test suite, 12 tests passed

Баримтын зорилго: Их сургуулийн хамгаалалт болон GitHub project танилцуулгад ашиглах professional software documentation.

Generated from project markdown documentation and styled HTML source.

Агуулга

1. Төслийн ерөнхий танилцуулга
2. Модулийн зорилго
3. Системийн архитектур
4. Folder structure
5. Ашигласан технологи
6. Authentication flow
7. OTP verification flow
8. API endpoints
9. Security tactics
10. Test results
11. Demo flow
12. Дүгнэлт

Screenshot placeholder: GitHub repository overview эсвэл project home screen

Энэхүү PDF нь репо-д байгаа `README.md`, `docs/MODULE_DOCUMENTATION.md`, `docs/API_ENDPOINTS.md`, `docs/SECURITY_TACTICS.md`, `docs/DEMO_GUIDE.md` баримтуудын агуулгыг нэгтгэн presentation-ready хэлбэрээр боловсруулсан.

1. Төслийн Ерөнхий Танилцуулга

Donation Auth Module нь “Хандивын систем”-ийн authentication болон authorization хэсгийг хариуцсан backend модуль юм.

Модуль нь хэрэглэгч бүртгэх, OTP баталгаажуулах, login хийх, JWT access token үүсгэх, refresh token ашиглах, logout хийх, profile харах болон role-based authorization хийх logic-ийг нэг дор төвлөрүүлсэн.

Системийн оролцогчид

Guest

Donor

Campaign Creator

Admin

Үндсэн боломжууд

- Register, OTP verify, Login
- JWT access token, refresh token
- Logout болон token invalidation
- Profile болон admin-only route

2. Модулийн Зорилго

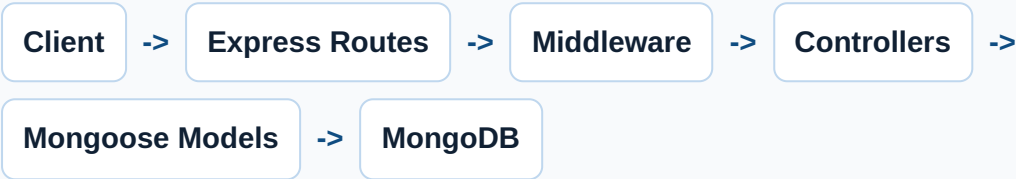
Модулийн зорилго нь хэрэглэгчийн identity-г найдвартай баталгаажуулж, системийн хамгаалагдсан endpoint-ууд руу зөвхөн зөв эрхтэй хэрэглэгч нэвтрэх боломжтой болгох юм.

- Бүртгэлийн үед email, phone, password, role validation хийх
- Password-г plain text хэлбэрээр хадгалахгүй байх
- OTP ашиглан хэрэглэгчийн бүртгэлийг баталгаажуулах
- Access token болон refresh token ашиглан session удирдах
- Role бүрийн permission-г backend middleware дээр шалгах

3. Системийн Архитектур

Модуль нь layered backend architecture ашигласан. Route layer нь endpoint-уудыг тодорхойлж, controller layer бизнес logic гүйцэтгэнэ. Middleware layer authentication, authorization, validation, security policy-г хэрэгжүүлнэ. Model layer нь MongoDB schema болон data persistence-г хариуцна.

Architecture Overview



Layer	Үүрэг	Файл/Жишээ
Application	Express app, global middleware, route mount	backend/src/app.js
Routes	API endpoint path болон validation/rate limiter холбох	authRoutes.js , userRoutes.js
Controllers	Register, OTP, login, refresh, logout business logic	authController.js
Middleware	JWT authentication, role authorization, validation	auth.js , validate.js
Data Model	User, OTP, refresh token schema	User.js

Screenshot placeholder: Architecture diagram эсвэл API client collection screenshot

4. Folder Structure

Project нь backend module болон documentation хэсгээс бүрдэнэ.

```
donation-auth-module/  
├── backend/  
│   ├── .env.example  
│   ├── package.json  
│   ├── package-lock.json  
│   └── src/  
│       ├── app.js  
│       ├── server.js  
│       ├── config/  
│       │   └── db.js  
│       ├── controllers/  
│       │   ├── adminController.js  
│       │   ├── authController.js  
│       │   └── userController.js  
│       ├── middleware/  
│       │   ├── auth.js  
│       │   ├── errorHandler.js  
│       │   └── validate.js  
│       ├── models/  
│       │   └── User.js  
│       ├── routes/  
│       │   ├── adminRoutes.js  
│       │   ├── authRoutes.js  
│       │   └── userRoutes.js  
│       ├── tests/  
│       │   └── auth.test.js  
│       └── utils/  
│           ├── otp.js  
│           ├── tokens.js  
│           └── validation.js  
├── docs/  
│   ├── API_ENDPOINTS.md  
│   ├── DEMO_GUIDE.md  
│   ├── MODULE_DOCUMENTATION.md  
│   ├── PROJECT_DOCUMENTATION.html  
│   ├── PROJECT_DOCUMENTATION.pdf  
│   └── SECURITY_TACTICS.md  
└── README.md
```

Гол файлууд

Файл	Тайлбар
src/app.js	Express app, helmet, cors, routes, error handler
src/models/User.js	User schema, OTP, refresh token, role field
src/controllers/authController.js	Register, OTP verify, login, refresh, logout logic
src/middleware/auth.js	JWT authentication болон role-based authorization
src/tests/auth.test.js	12 integration tests

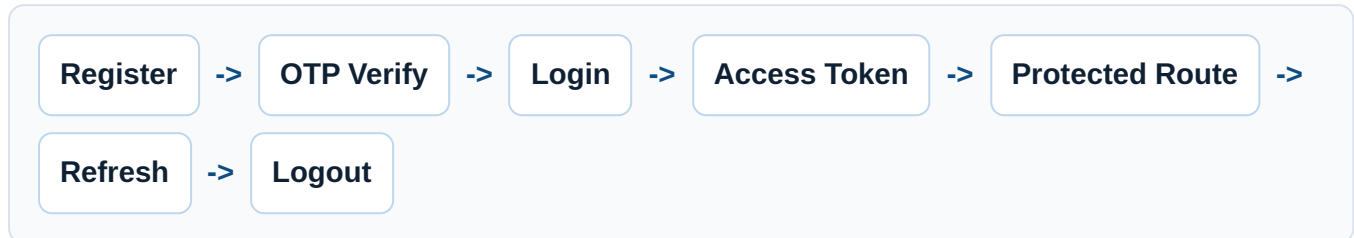
5. Ашигласан Технологи

Технологи	Үүрэг
Node.js	Backend runtime
Express	REST API framework
MongoDB	User, OTP, refresh token хадгалах database
Mongoose	MongoDB schema болон model layer
bcrypt	Password болон OTP hash хийх
jsonwebtoken	JWT access token болон refresh token үүсгэх
express-validator	Request input validation
express-rate-limit	Login, OTP, refresh endpoint rate limiting
helmet	HTTP security headers
cors	Frontend origin тохируулах
Jest	Automated test framework
Supertest	API integration testing
mongodb-memory-server	Test environment-д isolated MongoDB ажиллуулах

Dependency audit шалгалтаар production dependency vulnerability байхгүй болгож `bcrypt@6.0.0` ашигласан.

6. Authentication Flow

Authentication flow нь register, OTP verify, login, token usage, refresh, logout гэсэн үндсэн lifecycle-тэй.



1. User register хийнэ.
2. Server password-г bcrypt ашиглан hash хийнэ.
3. Server 6 оронтой OTP үүсгэж hash хэлбэрээр хадгална.
4. User OTP verify хийсний дараа `isVerified=true` болно.
5. User email/phone болон password ашиглан login хийнэ.
6. Server access token болон refresh token буцаана.
7. Protected route дээр `Authorization: Bearer <accessToken>` ашиглана.
8. Access token expire бол refresh token ашиглан шинэ token авна.
9. Refresh үед refresh token rotate хийгдэж хуучин token хүчингүй болно.
10. Logout үед refresh token database-аас устгагдана.

Login Request Example

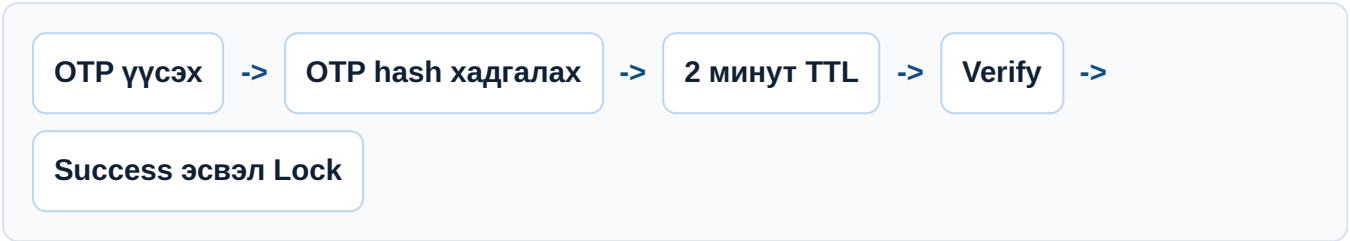
```
{
  "identifier": "donor@example.com",
  "password": "StrongPass123"
}
```

Login Response Example

```
{
  "message": "Login successful",
  "accessToken": "jwt_access_token",
  "refreshToken": "jwt_refresh_token",
  "user": {
    "email": "donor@example.com",
    "role": "donor",
    "isVerified": true
  }
}
```

7. OTP Verification Flow

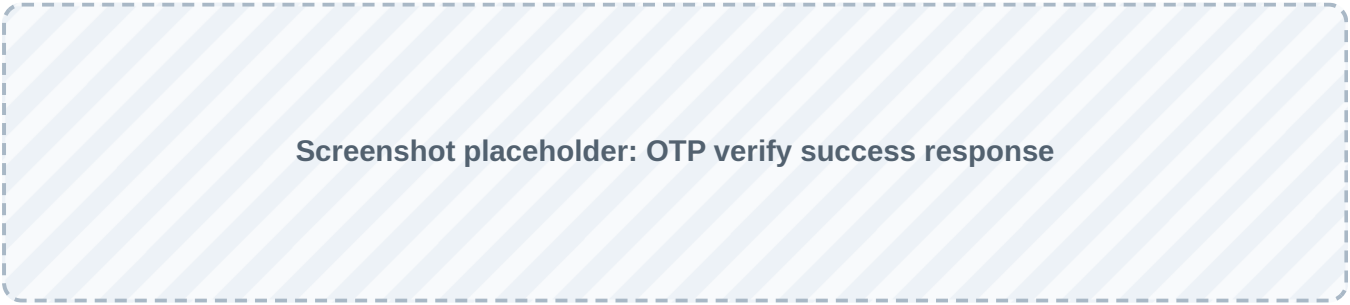
OTP verification нь бүртгүүлсэн хэрэглэгчийн email/phone эзэмшлийг баталгаажуулах зорилготой. OTP нь 6 оронтой, 2 минутын хүчинтэй хугацаатай, 3 удаа буруу оруулахад lock болно.



Policy	Утга	Зорилго
OTP length	6 digit	Хэрэглэгчид оруулахад энгийн, brute-force хамгаалалттай хосолно
Expiration	2 минут	Хуучин OTP ашиглах эрсдэлийг багасгана
Failed attempts	3 удаа	OTP таах оролдлогыг хязгаарлана
Storage	bcrypt hash	OTP raw value database-д хадгалагдахгүй

OTP Verify Request Example

```
{
  "identifier": "donor@example.com",
  "otp": "123456"
}
```



8. API Endpoints

API endpoint-ууд нь public, authenticated, admin-only access rule-тэй.

Method	Endpoint	Access	Тайлбар
GET	/health	Public	Server health шалгах
POST	/api/auth/register	Public	Хэрэглэгч бүртгэх, OTP үүсгэх
POST	/api/auth/verify-otp	Public, rate limited	OTP баталгаажуулах
POST	/api/auth/login	Public, rate limited	Email/phone + password login
POST	/api/auth/refresh	Public, rate limited	Refresh token rotate хийх
POST	/api/auth/logout	Public	Refresh token invalid хийх
GET	/api/users/me	Authenticated	Current user profile авах
GET	/api/admin/users	Admin only	Бүх user жагсаалт авах

Protected Route Header

```
Authorization: Bearer <accessToken>
```

Register Request Example

```
{
  "name": "Test Donor",
  "phone": "+97699112233",
  "email": "donor@example.com",
  "password": "StrongPass123",
  "role": "donor"
}
```

9. Security Tactics

Модуль нь authentication болон authorization хэсэгт шаардлагатай хамгаалалтын тактикуудыг backend түвшинд хэрэгжүүлсэн.

Security tactic	Хэрэгжүүлэлт	Хамгаалж буй эрсдэл
bcrypt password hashing	Password-г <code>bcrypt.hash</code> ашиглан хадгална	Plain text password leak
JWT access token	<code>sub</code> , <code>role</code> payload бүхий signed token	Unauthorized protected route access
Refresh token rotation	Refresh бүр дээр шинэ token үүсгэж хуучныг invalid болгоно	Refresh token reuse
Refresh token invalidation	Logout үед refresh token hash database-аас устгана	Logout дараах session reuse
OTP expiration	OTP 2 минут хүчинтэй	Хуучин OTP ашиглах
OTP attempt lock	3 буруу оролдлогын дараа lock	OTP brute-force
Input validation	<code>express-validator</code> ашигласан	Invalid input, bad request
Rate limiting	Login, OTP, refresh endpoint дээр limit тавьсан	Brute-force болон abuse
Helmet	<code>helmet()</code> global middleware	Security header дутуу байх
CORS	<code>CORS_ORIGIN</code> env тохиргоо	Untrusted origin access

Sensitive fields болох `password` , `otp.codeHash` , `refreshTokens` нь Mongoose schema дээр default query-д сонгогдохгүй байхаар `select: false` тохируулагдсан.

10. Test Results

Модуль дээр Jest, Supertest, mongodb-memory-server ашиглан integration test хийсэн.

Test summary: 1 test suite passed, 12 tests passed.

```
PASS src/tests/auth.test.js
Authentication and authorization module
  ✓ register success
  ✓ duplicate email
  ✓ invalid phone
  ✓ OTP verify success
  ✓ OTP expired
  ✓ OTP 3 failed attempts
  ✓ login success
  ✓ wrong password
  ✓ access protected route without token
  ✓ admin route blocked for non-admin
  ✓ refresh token rotates and invalidates old token
  ✓ logout invalidates refresh token
```

```
Test Suites: 1 passed
Tests:      12 passed
```

Test case	Шалгаж буй зүйл
register success	Зөв payload-оор хэрэглэгч үүсэх
duplicate email	Давхардсан email reject хийх
invalid phone	Phone validation ажиллах
OTP verify success	Зөв OTP user-г verified болгох
OTP expired	Expired OTP reject хийх
OTP 3 failed attempts	3 буруу оролдлогын дараа lock хийх
login success	Verified user token авах
wrong password	Буруу password reject хийх
protected route without token	Tokenгүй request 401 буцаах
admin route blocked for non-admin	Donor admin route ашиглах үед 403 буцаах
refresh token rotation	Old refresh token invalid болох
logout invalidation	Logout дараа refresh token хүчингүй болох

Screenshot placeholder: Terminal дээрх npm test 12 passed result

11. Demo Flow

Их сургуулийн хамгаалалт дээр дараах demo дараалал нь module-ийн гол functionality болон security behavior-ийг бүрэн харуулна.

1. Register request илгээж user үүсгэх.
2. Register response дээрх `dev0tp`-г ашиглан OTP verify хийх.
3. Login хийж `accessToken`, `refreshToken` авах.
4. `/api/users/me` protected route-г access token-той дуудах.
5. Donor token ашиглан `/api/admin/users` дуудаж `403 Forbidden` авах.
6. Refresh token rotate хийж шинэ token авах.
7. Хуучин refresh token-г дахин ашиглаж `401 Invalid refresh token` авах.
8. Logout хийж refresh token invalid болгох.

Demo command example

```
curl -X POST http://localhost:5000/api/auth/register \
-H "Content-Type: application/json" \
-d '{
  "name": "Test Donor",
  "phone": "+97699112233",
  "email": "donor@example.com",
  "password": "StrongPass123",
  "role": "donor"
}'
```

Security demo	Expected result
Invalid phone register	400 Validation failed
Duplicate email	409 already registered
Wrong OTP 3 удаа	423 locked
Wrong password login	401 Invalid credentials
Tokenгүй profile route	401 Access token is required
Donor token-р admin route	403 Forbidden
Refresh token reuse	401 Invalid refresh token
Logout дараа refresh	401 Invalid refresh token

Screenshot placeholder: Postman/Thunder Client demo collection screenshot

12. Дүгнэлт

Donation Auth Module нь онлайн хандивын системд шаардлагатай authentication болон authorization-ийн үндсэн шаардлагуудыг бүрэн хэрэгжүүлсэн.

Модуль нь register, OTP verification, login, JWT access token, refresh token rotation, logout, profile access, admin-only authorization зэрэг functionality-г аюулгүй байдлын тактикуудтай хослуулан хэрэгжүүлсэн.

Хэрэгжсэн гол шаардлага

- Register/Login/OTP
- JWT authentication
- Refresh token rotation
- Role-based authorization
- 12 automated tests

Хамгаалалтын үр дүн

- Password plain text хадгалагдахгүй
- OTP brute-force хязгаарлагдана
- Token reuse эрсдэл буурна
- Admin route role-оор хамгаалагдана

PDF generation approach: Markdown documentation агуулгыг presentation-ready HTML template болгон нэгтгэж, Google Chrome headless print engine ашиглан PDF болгосон.